

Ridge Lasso ElasticNet DT

You said:

very easy word Ridge, Lasso, ElasticNet, Robust Regression, Decision Tree Classification explain every topic and concept with code

ChatGPT said:

Ridge, Lasso, Elastic Net, Robust Regression and Decision Tree Classification

1. First understand regularization

A regression model learns this equation:

$$[\hat{y}] = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Where:

- $(\hat{y})$ = predicted value
- (b_0) = intercept
- $(b_1, b_2, \dots)$ = coefficients or weights
- $(x_1, x_2, \dots)$ = input features

A normal Linear Regression model tries to reduce prediction error.

Sometimes it learns the training data too closely. This is called **overfitting**.

Regularization adds a penalty for large coefficients:

| Prediction error + penalty

This keeps the model simpler and reduces overfitting.

2. Ridge Regression

Easy definition

Ridge Regression is Linear Regression with an **L2 penalty**.

It reduces large coefficients but normally does not make them exactly zero.

Formula

$$[\text{Ridge Loss}] = \text{MSE} + \alpha \sum_{j=1}^n b_j^2$$

It adds the square of every coefficient to the error.

Meaning of **alpha**

- **alpha = 0**: works almost like normal Linear Regression
- Small **alpha**: small penalty
- Large **alpha**: strong penalty
- Very large **alpha**: coefficients become very small and the model may underfit

Simple example

Suppose the coefficients are:

Experience coefficient = 100
Education coefficient = 5
Age coefficient = 80

Ridge reduces large coefficients:

Experience coefficient = 70
Education coefficient = 4
Age coefficient = 55

It normally keeps all features.

When should we use Ridge?

Use Ridge when:

- The model is overfitting
- Many features are useful
- Features are related to each other
- You do not want to remove features

- Coefficients are becoming too large

Ridge Regression code

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Load regression dataset
X, y = load_diabetes(return_X_y=True)

# Divide data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

# StandardScaler first scales the features
# Ridge then trains the model
model = make_pipeline(
    StandardScaler(),
    Ridge(alpha=1.0)
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

Selecting the best **alpha**

```
from sklearn.linear_model import RidgeCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

model = make_pipeline(
    StandardScaler(),
    RidgeCV(alphas=[0.001, 0.01, 0.1, 1, 10, 100])
)

model.fit(X_train, y_train)

ridge_model = model.named_steps["ridgecv"]

print("Best alpha:", ridge_model.alpha_)
print("Coefficients:", ridge_model.coef_)
```

Advantages

- Reduces overfitting
- Handles related features well
- Keeps all features
- Usually more stable than Linear Regression

Disadvantages

- Does not perform feature selection
- Scaling is important
- A very large **alpha** can cause underfitting

3. Lasso Regression

Easy definition

Lasso Regression is Linear Regression with an **L1 penalty**.

Lasso can make some coefficients exactly zero. Therefore, it can automatically remove less useful features.

Formula

$$[\text{Lasso Loss} = \text{MSE} + \alpha \sum_{j=1}^n |b_j|]$$

It adds the absolute value of each coefficient to the error.

Simple example

Before Lasso:

```
Experience = 80  
Education  = 2  
Age       = 25  
Height    = 1
```

After Lasso:

```
Experience = 65  
Education  = 0  
Age       = 15  
Height    = 0
```

Features with zero coefficients are ignored.

When should we use Lasso?

Use Lasso when:

- There are many features
- Some features are not useful
- You need automatic feature selection

- You want a simpler and more understandable model

Lasso Regression code

```
from sklearn.linear_model import Lasso
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

model = make_pipeline(
    StandardScaler(),
    Lasso(alpha=0.1, max_iter=10000)
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

lasso_model = model.named_steps["lasso"]

print("Coefficients:", lasso_model.coef_)
print("Number of selected features:",
      sum(lasso_model.coef_ != 0))
```

Selecting the best **alpha**

```
from sklearn.linear_model import LassoCV

model = make_pipeline(
    StandardScaler(),
    LassoCV(
        alphas=[0.001, 0.01, 0.1, 1, 10],
        cv=5,
        max_iter=10000
    )
)

model.fit(X_train, y_train)

lasso_model = model.named_steps["lassocv"]

print("Best alpha:", lasso_model.alpha_)
print("Coefficients:", lasso_model.coef_)
```

Advantages

- Reduces overfitting
- Performs automatic feature selection
- Produces a simpler model
- Helpful when the dataset has many columns

Disadvantages

- It may remove an important feature
 - It can behave poorly when many features are strongly related
 - Scaling is important
 - A large **alpha** may remove too many features
-

4. Elastic Net Regression

Easy definition

Elastic Net combines Ridge and Lasso.

It uses:

- L1 penalty from Lasso
- L2 penalty from Ridge

Therefore, it can reduce coefficients and also make some coefficients zero.

Formula

$$[\text{Elastic Net Loss}] = \text{MSE} + \alpha \left[r \sum |b_j| + (1-r) \sum b_j^2 \right]$$

Here, (r) is called **l1_ratio**.

Understanding **l1_ratio**

l1_ratio = 1.0 → works like Lasso
 l1_ratio = 0.0 → works like Ridge
 l1_ratio = 0.5 → 50% Lasso and 50% Ridge
 l1_ratio = 0.7 → more Lasso, less Ridge
 l1_ratio = 0.2 → less Lasso, more Ridge

When should we use Elastic Net?

Use Elastic Net when:

- There are many features
- Some features are not useful
- Some features are strongly related
- You need both coefficient reduction and feature selection
- Lasso removes too many related features

Elastic Net code

```
from sklearn.linear_model import ElasticNet
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

model = make_pipeline(
    StandardScaler(),
    ElasticNet(
        alpha=0.1,
        l1_ratio=0.5,
        max_iter=10000
    )
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

elastic_model = model.named_steps["elasticnet"]

print("Coefficients:", elastic_model.coef_)
```

Automatically selecting parameters

```
from sklearn.linear_model import ElasticNetCV

model = make_pipeline(
    StandardScaler(),
    ElasticNetCV(
        alphas=[0.001, 0.01, 0.1, 1, 10],
        l1_ratio=[0.1, 0.3, 0.5, 0.7, 0.9],
        cv=5,
        max_iter=10000
    )
)

model.fit(X_train, y_train)

elastic_model = model.named_steps["elasticnetcv"]

print("Best alpha:", elastic_model.alpha_)
print("Best l1_ratio:", elastic_model.l1_ratio_)
print("Coefficients:", elastic_model.coef_)
```

Advantages

- Combines Ridge and Lasso
- Performs feature selection
- Works better with related features than Lasso
- Reduces overfitting

Disadvantages

- Two parameters must be selected
- More complex than Ridge or Lasso
- Scaling is important
- Training may take more time

5. Ridge vs Lasso vs Elastic Net

Point	Ridge	Lasso	Elastic Net
Penalty	L2	L1	L1 + L2
Makes coefficients smaller	Yes	Yes	Yes

Point	Ridge	Lasso	Elastic Net
Makes coefficients zero	Usually no	Yes	Yes
Feature selection	No	Yes	Yes
Handles related features	Good	May select only one	Very good
Main parameters	alpha	alpha	alpha, l1_ratio
Best use	Most features are useful	Few features are useful	Many related features

Easy memory trick

- **Ridge:** Reduces coefficients
- **Lasso:** Leaves only useful features
- **Elastic Net:** Uses both Ridge and Lasso

Important point

Always scale the features before Ridge, Lasso and Elastic Net because their penalties depend on coefficient size.

Use this:

```
make_pipeline(
    StandardScaler(),
    Ridge()
)
```

6. Robust Regression

Easy definition

Robust Regression is a regression method that is less affected by **outliers**.

An outlier is a value that is very different from most other values.

Example:

Salaries: 30,000, 32,000, 31,000, 29,000, 10,00,000

Here, 10,00,000 is an outlier.

Normal Linear Regression may move its regression line toward this extreme value. Robust Regression tries to reduce the outlier's effect.

Main robust regression methods

A. Huber Regression

Huber Regression:

- Uses squared error for normal points
- Uses a smaller, linear penalty for very large errors
- Does not completely ignore outliers
- Reduces their effect

Huber Loss

For a small error:

$$[\text{Loss} = \frac{1}{2}e^2]$$

For a large error, the loss grows more slowly.

Here:

$$[e = y - \hat{y}]$$

Huber code

```
from sklearn.linear_model import HuberRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

huber_model = make_pipeline(
    StandardScaler(),
    HuberRegressor(
        epsilon=1.35,
        alpha=0.0001,
        max_iter=1000
    )
)

huber_model.fit(X_train, y_train)

y_pred = huber_model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

Important Huber parameters

- **epsilon**: decides which errors should be treated like outliers
 - Smaller **epsilon**: model is more sensitive to outliers
 - Larger **epsilon**: model behaves more like normal Linear Regression
 - **alpha**: regularization strength
 - **max_iter**: maximum training iterations
-

B. RANSAC Regression

RANSAC means:

Random Sample Consensus

How it works

1. Randomly selects a small group of data points.
2. Trains a model using those points.
3. Checks which other points are close to that model.
4. Close points are called **inliers**.
5. Far points are called **outliers**.
6. Repeats the process.
7. Keeps the model with the most inliers.

RANSAC code

```
from sklearn.linear_model import LinearRegression, RANSACRegressor

ransac_model = RANSACRegressor(
    estimator=LinearRegression(),
    min_samples=0.5,
    residual_threshold=None,
    max_trials=100,
    random_state=42
)

ransac_model.fit(X_train, y_train)

y_pred = ransac_model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

print("Training inliers:", ransac_model.inlier_mask_)
```


Important RANSAC parameters

- **estimator**: base model, such as `LinearRegression()`
- **min_samples**: minimum data used for one trial
- **residual_threshold**: maximum error for an inlier
- **max_trials**: number of random trials
- **random_state**: produces repeatable results

When to use RANSAC

Use it when:

- The data has clear, strong outliers
- Outliers should be ignored
- Most data points follow one pattern

C. Theil-Sen Regression

Theil-Sen calculates slopes using many pairs of data points and takes a middle or median result.

It is resistant to outliers but can be slow on large datasets.

```
from sklearn.linear_model import TheilSenRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

theil_model = make_pipeline(
    StandardScaler(),
    TheilSenRegressor(random_state=42)
)

theil_model.fit(X_train, y_train)

y_pred = theil_model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

Robust methods comparison

Method	Outlier handling	Best situation
Huber	Reduces outlier effect	Moderate outliers
RANSAC	Separates inliers and outliers	Strong, clear outliers

Method	Outlier handling	Best situation
Theil-Sen	Uses median-based estimation	Small or medium datasets

Important warning

Do not automatically remove every outlier. An outlier can be:

- A data-entry mistake
- A measurement error
- A real but rare observation
- An important business event

First investigate why it exists.

7. Decision Tree Classification

Easy definition

Decision Tree Classification is a supervised learning algorithm that predicts a category using a series of questions.

Example:

```

Is age greater than 30?
├── Yes: Is income greater than 50,000?
│   ├── Yes: Buy
│   └── No: Not Buy
└── No: Not Buy
  
```

It works like a flowchart.

Main terms

Root node

The first question at the top of the tree.

Decision node

A point where the tree asks a question.

Branch

The path created by the answer to a question.

Leaf node

The final prediction.

Split

Dividing the data using a condition.

Parent node

A node that is divided.

Child node

A node created after a split.

Tree depth

The number of levels in the tree.

8. How does a Decision Tree select a question?

The tree tries different questions, such as:

Age ≤ 30 ?
Salary $\leq 50,000$?
Experience ≤ 2 ?

It selects the question that separates the classes most clearly.

The two common methods are:

- Gini impurity
- Entropy

Gini impurity

[$Gini = 1 - \sum p_i^2$]

Where (p_i) is the proportion of each class.

Meaning

- **Gini = 0**: completely pure node
- Higher Gini: classes are mixed

Example:

Node A: 10 Yes, 0 No → pure
Node B: 5 Yes, 5 No → mixed

Node A has a lower Gini value.

Entropy

[$\text{Entropy} = -\sum p_i \log_2(p_i)$]

Meaning

- **Entropy = 0**: completely pure
- High entropy: classes are mixed

Information gain

Information gain shows how much impurity was removed by a split.

[$\text{Information Gain} = \text{Parent Impurity} - \text{Weighted Child Impurity}$]

The tree prefers the split with the highest information gain.

9. Decision Tree Classification code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

# Load classification data
iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    min_samples_split=2,
    min_samples_leaf=1,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)
```

Predicting one new flower

```
new_flower = [[5.1, 3.5, 1.4, 0.2]]

prediction = model.predict(new_flower)
probability = model.predict_proba(new_flower)

print("Predicted class:", iris.target_names[prediction[0]])
print("Class probabilities:", probability)
```

10. Important Decision Tree parameters

criterion

Method used to measure split quality.

```
criterion="gini"
```

or:

```
criterion="entropy"
```

max_depth

Maximum number of levels.

```
max_depth=3
```

- Small value: simpler tree
- Large value: complex tree
- No limit: high chance of overfitting

min_samples_split

Minimum samples required to divide a node.

```
min_samples_split=5
```

A node with fewer samples will not split.

min_samples_leaf

Minimum samples required inside a leaf.

```
min_samples_leaf=3
```

Increasing it normally creates a simpler tree.

max_features

Maximum features checked when finding a split.

```
max_features="sqrt"
```

class_weight

Helpful for unbalanced classes.

```
class_weight="balanced"
```

It gives more importance to the smaller class.

ccp_alpha

Controls post-pruning.

```
ccp_alpha=0.01
```

A larger value removes more branches.

11. Displaying tree rules

```
from sklearn.tree import export_text

rules = export_text(
    model,
    feature_names=list(iris.feature_names)
)

print(rules)
```

Example output:

```
petal length <= 2.45
|--- class: setosa
petal length > 2.45
|--- petal width <= 1.75
|   |--- class: versicolor
|   |--- petal width > 1.75
|       |--- class: virginica
```

12. Visualizing the tree

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(14, 8))

plot_tree(
    model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True,
    rounded=True
)

plt.show()
```

13. Feature importance

Feature importance tells us which columns contributed more to predictions.

```
for name, importance in zip(
    iris.feature_names,
    model.feature_importances_
):
    print(name, ":", importance)
```

Important caution:

Feature importance shows that a feature was useful to the tree. It does not prove that the feature caused the result.

14. Overfitting in Decision Trees

A fully grown tree may memorize the training data.

Example:

```
Training accuracy = 100%  
Testing accuracy = 75%
```

This is overfitting.

Controlling overfitting

Use:

```
DecisionTreeClassifier(  
    max_depth=4,  
    min_samples_split=10,  
    min_samples_leaf=5,  
    ccp_alpha=0.01,  
    random_state=42  
)
```

This is called **pruning**.

Pre-pruning

Stop the tree from growing too much:

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`
- `max_leaf_nodes`

Post-pruning

First grow the tree and then remove weak branches:

- `ccp_alpha`
-

15. Finding the best Decision Tree parameters

```
from sklearn.model_selection import GridSearchCV

parameter_grid = {
    "criterion": ["gini", "entropy"],
    "max_depth": [2, 3, 4, 5, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 5],
    "ccp_alpha": [0.0, 0.001, 0.01]
}

search = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_grid=parameter_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

search.fit(X_train, y_train)

print("Best parameters:", search.best_params_)
print("Best validation score:", search.best_score_)

best_tree = search.best_estimator_

y_pred = best_tree.predict(X_test)

print("Test accuracy:", accuracy_score(y_test, y_pred))
```

16. Advantages of Decision Tree

- Very easy to understand
- Works for classification and regression
- Does not require feature scaling
- Can learn non-linear patterns
- Handles numerical and encoded categorical features
- Provides understandable rules
- Can show feature importance

17. Disadvantages of Decision Tree

- Can overfit easily
- A small data change may create a different tree

- A single tree may not give the best accuracy
- It may favour features with many possible split points
- Deep trees become difficult to understand

Random Forest and boosting models improve many of these problems by combining multiple trees.

18. Final interview comparison

Algorithm	Main purpose	Outlier resistant	Feature selection	Scaling needed
Ridge	Reduce overfitting	No	No	Yes
Lasso	Reduce overfitting and remove features	No	Yes	Yes
Elastic Net	Combine Ridge and Lasso	No	Yes	Yes
Huber	Regression with moderate outliers	Yes	No	Recommended
RANSAC	Ignore strong outliers	Yes	No	Depends on base model
Decision Tree	Predict categories using rules	Reasonably resistant	Uses important splits	No

One-line interview answers

- **Ridge:** Linear Regression with an L2 penalty that reduces coefficients but keeps all features.
- **Lasso:** Linear Regression with an L1 penalty that can make unnecessary feature coefficients zero.
- **Elastic Net:** A combination of Ridge and Lasso penalties.
- **Robust Regression:** Regression designed to reduce the effect of outliers.
- **Decision Tree Classification:** A flowchart-like model that predicts a class by asking feature-based questions.

You said:

very easy word Ridge, Lasso, ElasticNet, Robust Regression, Decision Tree Classification explain every topic and concept with code real life

ChatGPT said:

Ridge, Lasso, Elastic Net, Robust Regression and Decision Tree Classification

These algorithms solve two different problems:

Algorithm	Problem type	Real-life example
Ridge	Regression	Predict house price
Lasso	Regression	Predict house price and remove useless columns
Elastic Net	Regression	Predict price when features are related
Robust Regression	Regression	Predict salary when data contains outliers
Decision Tree Classification	Classification	Approve or reject a loan

- **Regression** predicts a number: price, salary, sales.
- **Classification** predicts a category: yes/no, spam/not spam.

Part 1: Ridge, Lasso and Elastic Net

1. What problem do they solve?

Suppose we want to predict house price using:

- House size
- Number of rooms
- House age
- Distance from city
- Parking spaces

Linear Regression learns:

$$[\text{Price} = b_0 + b_1(\text{Size}) + b_2(\text{Rooms}) + b_3(\text{Age}) + \dots]$$

The values ($b_1, b_2, \dots$) are called **coefficients** or **weights**.

Sometimes Linear Regression gives very large coefficients and memorizes the training data. This is called **overfitting**.

What is regularization?

Regularization adds a penalty for large coefficients.

Final Loss = Prediction Error + Penalty

It tells the model:

“Make good predictions, but do not make the model unnecessarily complicated.”

Ridge, Lasso and Elastic Net are regularization methods.

2. Ridge Regression

Easy definition

Ridge Regression is Linear Regression with an **L2 penalty**.

It makes coefficients smaller, but normally does not make them exactly zero.

Real-life example

Suppose house size and number of rooms are strongly related:

Large house → usually more rooms
Small house → usually fewer rooms

Normal Linear Regression may become confused because both columns provide similar information.

Ridge handles these related features well. It keeps both features but reduces their coefficients.

Formula

[$\text{Loss} = \text{MSE} + \alpha \sum b_j^2$]

Easy meaning:

Loss = Prediction mistakes + Square of coefficients

What is **alpha**?

alpha controls the penalty.

Alpha	Result
0	Almost normal Linear Regression
Small value	Small penalty
Large value	Strong penalty

Alpha	Result
Too large	Model becomes too simple and may underfit

Ridge real-life code

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, r2_score

# House-price data
data = {
    "size_sqft": [
        800, 900, 1000, 1100, 1200,
        1300, 1400, 1500, 1600, 1700,
        1800, 1900, 2000, 2100, 2200
    ],
    "rooms": [
        2, 2, 2, 3, 3,
        3, 3, 4, 4, 4,
        4, 5, 5, 5, 6
    ],
    "age_years": [
        20, 18, 17, 15, 14,
        12, 11, 10, 9, 8,
        7, 6, 5, 4, 2
    ],
    "distance_km": [
        15, 14, 13, 12, 11,
        10, 9, 8, 7, 6,
        5, 4, 3, 2, 1
    ],
    "price_lakh": [
        25, 28, 31, 35, 39,
        43, 47, 52, 57, 62,
        68, 74, 81, 88, 96
    ]
}

df = pd.DataFrame(data)

# Input columns
X = df[
    ["size_sqft", "rooms", "age_years", "distance_km"]
]
```

```
# Output column
y = df["price_lakh"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

# Scaling is important for Ridge
model = make_pipeline(
    StandardScaler(),
    Ridge(alpha=1.0)
)

model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("Predictions:", predictions)
print("Actual prices:", y_test.values)
print("MAE:", mean_absolute_error(y_test, predictions))
print("R2 score:", r2_score(y_test, predictions))
```

Predicting a new house

```
new_house = pd.DataFrame({
    "size_sqft": [1750],
    "rooms": [4],
    "age_years": [6],
    "distance_km": [5]
})

predicted_price = model.predict(new_house)

print("Predicted house price:",
      round(predicted_price[0], 2),
      "lakh")
```

Advantages

- Reduces overfitting
- Handles related features well
- Keeps every feature

- Produces stable predictions

Disadvantages

- Does not remove useless features
- Feature scaling is important
- A very large **alpha** can cause underfitting

When should we use Ridge?

Use Ridge when:

- Most features are useful
- Features are related
- Linear Regression is overfitting
- You want to keep all features

3. Lasso Regression

Easy definition

Lasso Regression is Linear Regression with an **L1 penalty**.

It can make some coefficients exactly zero.

If a coefficient becomes zero, the model stops using that feature. Therefore, Lasso can perform **feature selection**.

Real-life example

Suppose we predict house price using:

- House size
- Rooms
- Age
- Distance
- House ID number

The house ID usually does not affect the price.

Lasso can give it a coefficient of zero:

House size coefficient = 12
Rooms coefficient = 5
Age coefficient = -2
House ID coefficient = 0

The model removes the useless `house_id` feature.

Formula

[$\text{Loss} = \text{MSE} + \alpha \sum |b_j|$]

Easy meaning:

Loss = Prediction mistakes + Absolute value of coefficients

Lasso real-life code

```
from sklearn.linear_model import Lasso

# Add a mostly useless feature
df["house_number"] = [
    101, 205, 309, 412, 518,
    623, 727, 832, 936, 1041,
    1146, 1250, 1355, 1460, 1564
]

X = df[
    [
        "size_sqft",
        "rooms",
        "age_years",
        "distance_km",
        "house_number"
    ]
]

y = df["price_lakh"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

lasso_model = make_pipeline(
    StandardScaler(),
    Lasso(
        alpha=0.5,
        max_iter=10000
    )
)

lasso_model.fit(X_train, y_train)

predictions = lasso_model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, predictions))
print("R2 score:", r2_score(y_test, predictions))

# Get coefficients from the trained Lasso model
trained_lasso = lasso_model.named_steps["lasso"]

for feature, coefficient in zip(
    X.columns,
```

```
trained_lasso.coef_  
):  
    print(feature, ":", coefficient)
```

A feature with coefficient 0 has been removed by Lasso.

Advantages

- Reduces overfitting
- Removes less useful features
- Makes the model simpler
- Helpful when the dataset contains many columns

Disadvantages

- It may remove an important feature
- It can be unstable when features are strongly related
- Feature scaling is important
- A large **alpha** can remove too many features

When should we use Lasso?

Use Lasso when:

- There are many features
- Some features are useless
- You want automatic feature selection
- You want a simpler model

4. Elastic Net Regression

Easy definition

Elastic Net combines Ridge and Lasso.

Elastic Net = Ridge + Lasso

It can:

- Reduce large coefficients like Ridge
- Make useless coefficients zero like Lasso

- Handle related features better than Lasso

Real-life example

Consider these house features:

House size
Number of bedrooms
Number of rooms
Covered area

These features may be strongly related.

Lasso may keep one feature and remove the others. Ridge keeps all features.

Elastic Net finds a balance:

- It keeps useful related features
- It removes useless features
- It reduces large coefficients

Formula

[Loss=MSE+ $\alpha \left[l_1_ratio \sum |b_j| + (1-l_1_ratio) \sum b_j^2 \right]$]

You only need to remember:

Elastic Net Loss =
Error + Lasso penalty + Ridge penalty

Understanding l_1_ratio

l_1_ratio	Meaning
1.0	Works like Lasso
0.0	Works like Ridge
0.5	Half Lasso and half Ridge
0.8	More Lasso

l1_ratio **Meaning**

0.2 More Ridge

Elastic Net real-life code

```
from sklearn.linear_model import ElasticNet

elastic_model = make_pipeline(
    StandardScaler(),
    ElasticNet(
        alpha=0.5,
        l1_ratio=0.5,
        max_iter=10000
    )
)

elastic_model.fit(X_train, y_train)

predictions = elastic_model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, predictions))
print("R2 score:", r2_score(y_test, predictions))

trained_elastic = elastic_model.named_steps["elasticnet"]

for feature, coefficient in zip(
    X.columns,
    trained_elastic.coef_
):
    print(feature, ":", coefficient)
```

Important parameters

alpha

Controls the total penalty.

Large alpha → stronger penalty
Small alpha → weaker penalty

l1_ratio

Controls how much Lasso and Ridge are used.

l1_ratio near 1 $\rightarrow$ more Lasso

l1_ratio near 0 $\rightarrow$ more Ridge

Automatically finding the best values

```
from sklearn.linear_model import ElasticNetCV

elastic_cv_model = make_pipeline(
    StandardScaler(),
    ElasticNetCV(
        alphas=[0.001, 0.01, 0.1, 0.5, 1, 10],
        l1_ratio=[0.1, 0.3, 0.5, 0.7, 0.9],
        cv=5,
        max_iter=10000
    )
)

elastic_cv_model.fit(X_train, y_train)

trained_model = elastic_cv_model.named_steps["elasticnetcv"]

print("Best alpha:", trained_model.alpha_)
print("Best l1_ratio:", trained_model.l1_ratio_)
```

Advantages

- Combines Ridge and Lasso
- Handles related features
- Performs feature selection
- Reduces overfitting

Disadvantages

- Has two important parameters
- More complex than Ridge and Lasso
- Feature scaling is important
- Can take more time to train

5. Ridge vs Lasso vs Elastic Net

Point	Ridge	Lasso	Elastic Net
Penalty	L2	L1	L1 + L2

Point	Ridge	Lasso	Elastic Net
Makes coefficients smaller	Yes	Yes	Yes
Makes coefficients zero	Usually no	Yes	Yes
Removes features	No	Yes	Yes
Related features	Handles well	Can remove some	Handles well
Parameters	alpha	alpha	alpha, l1_ratio
Best situation	Most features are useful	Some features are useless	Features are related and some are useless

Easy memory trick

- **Ridge:** Reduces coefficients
- **Lasso:** Removes useless features
- **Elastic Net:** Does both

Part 2: Robust Regression

6. What is an outlier?

An outlier is a value that is very different from the other values.

Example salary data:

Experience: 1, 2, 3, 4, 5 years
Salary: 3, 4, 5, 6, 50 lakh

The salary **50 lakh** is very different. It may be:

- A data-entry mistake
- A special employee
- A company owner
- A genuine rare case

Normal Linear Regression can be pulled toward this unusual value.

Easy definition

Robust Regression is a regression method that reduces the effect of outliers.

Real-life example

A company wants to predict salary from years of experience.

Most employees follow a normal salary pattern, but one director has a very high salary.

Robust Regression focuses more on the normal employee pattern.

7. Huber Regression

Easy definition

Huber Regression treats:

- Small errors normally
- Very large errors more carefully

It does not fully remove outliers. It only reduces their effect.

Huber real-life code

```
import numpy as np

from sklearn.linear_model import LinearRegression, HuberRegressor
from sklearn.metrics import mean_absolute_error

# Years of experience
X = np.array([
    [1], [2], [3], [4], [5],
    [6], [7], [8], [9], [10]
])

# Salary in lakh
# 50 is an outlier
y = np.array([
    3, 4, 5, 6, 7,
    8, 9, 10, 11, 50
])

linear_model = LinearRegression()
linear_model.fit(X, y)

huber_model = HuberRegressor(
    epsilon=1.35,
    max_iter=1000
)
huber_model.fit(X, y)

new_employee = [[6]]

print(
    "Linear Regression prediction:",
    linear_model.predict(new_employee)
)

print(
    "Huber Regression prediction:",
    huber_model.predict(new_employee)
)
```

Linear Regression is affected more by the **50 lakh** outlier. Huber stays closer to the normal salary pattern.

Important Huber parameters

epsilon

Decides when an error should be treated as unusually large.

- Smaller value: treats more points as outliers
- Larger value: behaves more like Linear Regression

alpha

Controls regularization.

max_iter

Maximum number of training steps.

8. RANSAC Regression

Full form

RANSAC = Random Sample Consensus

Easy working

RANSAC follows these steps:

1. Randomly selects some data points.
2. Trains a regression line.
3. Finds points close to that line.
4. Calls close points **inliers**.
5. Calls far points **outliers**.
6. Repeats the process.

7. Selects the best line.

RANSAC real-life code

```
from sklearn.linear_model import RANSACRegressor

ransac_model = RANSACRegressor(
    estimator=LinearRegression(),
    min_samples=5,
    residual_threshold=2.0,
    max_trials=100,
    random_state=42
)

ransac_model.fit(X, y)

prediction = ransac_model.predict([[6]])

print("Predicted salary:", prediction[0])

print("Inliers:", ransac_model.inlier_mask_)
print("Outliers:", ~ransac_model.inlier_mask_)
```

Important parameters

- **estimator**: base regression model
- **min_samples**: minimum points used in one trial
- **residual_threshold**: maximum error allowed for an inlier
- **max_trials**: number of random attempts
- **random_state**: keeps results repeatable

When should we use RANSAC?

Use it when:

- Data has strong outliers
- Most points follow one clear pattern
- You want to separate outliers from normal points

9. Theil-Sen Regression

Easy definition

Theil-Sen finds slopes using many pairs of data points. It then uses the middle or median result.

Because it uses the median, extreme values affect it less.

```

from sklearn.linear_model import TheilSenRegressor

theil_model = TheilSenRegressor(random_state=42)

theil_model.fit(X, y)

prediction = theil_model.predict([[6]])

print("Predicted salary:", prediction[0])

```

Robust Regression comparison

Method	What it does	Best use
Huber	Reduces outlier effect	Medium outliers
RANSAC	Separates outliers and inliers	Strong outliers
Theil-Sen	Uses median-like slope	Small datasets

Important interview point

Ridge and Lasso are mainly used to reduce overfitting. They are not specially designed to handle outliers.

Robust Regression is specially designed for outlier-heavy data.

Part 3: Decision Tree Classification

10. Easy definition

Decision Tree Classification predicts a category by asking a series of questions.

Real-life loan example

A bank may ask:

```

Is credit score greater than 700?
├── Yes
│   ├── Is income greater than ₹50,000?
│   │   ├── Yes → Approve loan
│   │   └── No  → Reject loan
│   └── No  → Reject loan

```

It looks like an upside-down tree.

11. Important Decision Tree words

Root node

The first question.

Is credit score greater than 700?

Decision node

A middle question.

Is income greater than ₹50,000?

Branch

The path created by an answer.

Yes branch
No branch

Leaf node

The final result.

Loan Approved
Loan Rejected

Split

Dividing data using a question.

Parent node

The node before a split.

Child node

The new node created after a split.

Depth

The number of levels in the tree.

12. How does the tree choose a question?

The tree checks different questions:

Income $\leq 50,000$?
Credit score ≤ 700 ?
Previous default ≤ 0 ?
Existing loan ≤ 1 ?

It selects the question that separates approved and rejected customers most clearly.

The main split measurements are:

- Gini impurity
 - Entropy
-

13. Gini impurity

Gini checks how mixed a node is.

$$[\text{Gini} = 1 - \sum p_i^2]$$

Easy meaning:

- **Gini = 0**: all records belong to one class
- Higher Gini: classes are mixed

Example:

Node A: 10 Approved, 0 Rejected → Pure
Node B: 5 Approved, 5 Rejected → Mixed

The tree prefers purer nodes.

14. Entropy

Entropy also checks how mixed a node is.

$$[\text{Entropy} = -\sum p_i \log_2(p_i)]$$

Easy meaning:

- Entropy = 0: completely pure
- High entropy: classes are mixed

Both Gini and Entropy usually produce similar trees.

15. Information gain

Information gain checks how much a split improved the data.

[Information\ Gain = Parent\ Impurity-Child\ Impurity]

A good question gives high information gain.

16. Loan approval code

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

loan_data = {
    "income": [
        30000, 35000, 40000, 45000, 50000,
        55000, 60000, 65000, 70000, 75000,
        80000, 85000, 90000, 95000, 100000,
        42000, 58000, 72000, 88000, 110000
    ],
    "credit_score": [
        550, 580, 600, 620, 640,
        660, 680, 700, 720, 740,
        760, 780, 800, 820, 840,
        610, 690, 730, 790, 850
    ],
    "existing_loan": [
        2, 2, 1, 2, 1,
        1, 1, 0, 1, 0,
        0, 0, 0, 0, 0,
        2, 1, 0, 0, 0
    ],
    "previous_default": [
        1, 1, 1, 0, 1,
        0, 0, 0, 0, 0,
        0, 0, 0, 0, 0,
        1, 0, 0, 0, 0
    ],
    "loan_approved": [
        0, 0, 0, 0, 0,
        0, 1, 1, 1, 1,
        1, 1, 1, 1, 1,
        0, 1, 1, 1, 1
    ]
}

df = pd.DataFrame(loan_data)

X = df[
    "income",
```

```
        "credit_score",
        "existing_loan",
        "previous_default"
    ]
]

y = df["loan_approved"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

tree_model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    min_samples_split=2,
    min_samples_leaf=1,
    random_state=42
)

tree_model.fit(X_train, y_train)

predictions = tree_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, predictions))

print("Confusion matrix:")
print(confusion_matrix(y_test, predictions))

print("Classification report:")
print(classification_report(y_test, predictions))
```

17. Predicting loan approval for a new customer

```
new_customer = pd.DataFrame({
    "income": [70000],
    "credit_score": [750],
    "existing_loan": [0],
    "previous_default": [0]
})

prediction = tree_model.predict(new_customer)
probability = tree_model.predict_proba(new_customer)

if prediction[0] == 1:
    print("Loan Approved")
else:
    print("Loan Rejected")

print("Probabilities:", probability)
```

Example probability:

```
[[0.20, 0.80]]
```

It means:

```
Rejection probability = 20%
Approval probability = 80%
```

18. Important Decision Tree parameters

criterion

Method used to select a split:

```
criterion="gini"
```

or:

```
criterion="entropy"
```

max_depth

Maximum tree levels:

```
max_depth=3
```

- Small depth: simple tree
- Large depth: complex tree
- Very large depth: possible overfitting

min_samples_split

Minimum records needed to split a node:

```
min_samples_split=5
```

min_samples_leaf

Minimum records that must remain in a leaf:

```
min_samples_leaf=2
```

A larger value creates a simpler tree.

max_leaf_nodes

Maximum number of final leaves:

```
max_leaf_nodes=10
```

class_weight

Helpful when classes are unbalanced:

```
class_weight="balanced"
```

Example:

```
950 loan approvals  
50 loan rejections
```

The classes are unbalanced, so `class_weight="balanced"` gives more importance to the smaller class.

ccp_alpha

Used for pruning:

```
ccp_alpha=0.01
```

A larger value removes more unnecessary branches.

19. Overfitting in Decision Tree

Suppose:

```
Training accuracy = 100%  
Testing accuracy = 70%
```

The tree memorized training data but did not understand the general pattern. This is overfitting.

Pre-pruning

Stop the tree before it grows too much:

```
DecisionTreeClassifier(  
    max_depth=4,  
    min_samples_split=10,  
    min_samples_leaf=5,  
    max_leaf_nodes=10  
)
```

Post-pruning

Grow the tree and then remove weak branches:

```
DecisionTreeClassifier(  
    ccp_alpha=0.01  
)
```

20. Displaying tree rules

```
from sklearn.tree import export_text

rules = export_text(
    tree_model,
    feature_names=list(X.columns)
)

print(rules)
```

Possible rules:

```
credit_score <= 670
|--- loan rejected

credit_score > 670
|--- previous_default <= 0
|   |--- loan approved
|--- previous_default > 0
|   |--- loan rejected
```

21. Visualizing the tree

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(14, 8))

plot_tree(
    tree_model,
    feature_names=X.columns,
    class_names=["Rejected", "Approved"],
    filled=True,
    rounded=True
)

plt.show()
```

22. Feature importance

Feature importance shows which columns the tree used most.

```
importance_table = pd.DataFrame({
    "Feature": X.columns,
    "Importance": tree_model.feature_importances_
})

importance_table = importance_table.sort_values(
    by="Importance",
    ascending=False
)

print(importance_table)
```

Important:

Feature importance shows usefulness for prediction. It does not prove that a feature caused loan approval.

23. Decision Tree advantages and disadvantages

Advantages

- Easy to understand
- Works like real-life questions
- Does not need feature scaling
- Can learn non-linear relationships
- Can predict multiple classes
- Shows feature importance

Disadvantages

- Can overfit easily
- A small data change can change the tree
- A deep tree becomes difficult to understand
- One tree may be less accurate than Random Forest
- Biased training data can produce unfair decisions

For real bank decisions, sensitive personal features and fairness rules must be handled carefully.

24. Evaluation metrics

Regression metrics

MAE

Average absolute prediction mistake:

$$[\text{MAE} = \frac{\sum |\text{Actual} - \text{Predicted}|}{n}]$$

Lower MAE is better.

MSE

Average squared mistake:

$$[\text{MSE} = \frac{\sum (\text{Actual} - \text{Predicted})^2}{n}]$$

Lower MSE is better. It gives a larger punishment to big mistakes.

(R²) score

Shows how well the model explains the result:

R² near 1 → very good
R² near 0 → weak model
R² below 0 → very poor model

Classification metrics

- **Accuracy:** total correct predictions
 - **Precision:** out of predicted approvals, how many were correct
 - **Recall:** out of actual approvals, how many were found
 - **F1-score:** balance between precision and recall
 - **Confusion matrix:** table of correct and incorrect results
-

25. Final interview answers

What is Ridge Regression?

Ridge is Linear Regression with an L2 penalty. It reduces large coefficients and overfitting but normally keeps all features.

What is Lasso Regression?

Lasso is Linear Regression with an L1 penalty. It can make useless feature coefficients zero.

What is Elastic Net?

Elastic Net combines Ridge and Lasso. It reduces coefficients, removes useless features and handles related features.

What is Robust Regression?

Robust Regression reduces the effect of unusual or extreme data points called outliers.

What is Decision Tree Classification?

It is a classification algorithm that predicts a category by asking feature-based questions, like a flowchart.

Does a Decision Tree require scaling?

No. A Decision Tree compares feature values using conditions, so scaling is normally unnecessary.

Which model should we select?

Most features useful	→ Ridge
Many useless features	→ Lasso
Related and useless features	→ Elastic Net
Data contains strong outliers	→ Robust Regression
Need yes/no or category prediction	→ Decision Tree Classification

You said:

very easy word Ridge, Lasso, ElasticNet, Robust Regression, Decision Tree Classification explain every topic and concept with code real life formula with parameter with example and code and point for interview

ChatGPT said:

Complete Interview Notes: Ridge, Lasso, Elastic Net, Robust Regression and Decision Tree Classification

1. First understand the problem types

Regression

Regression predicts a number.

Examples:

- House price

- Employee salary
- Monthly sales
- Temperature
- Crop production

Classification

Classification predicts a category.

Examples:

- Loan approved or rejected
- Spam or not spam
- Fraud or genuine
- Disease or no disease

Algorithm	Problem type
Ridge	Regression
Lasso	Regression
Elastic Net	Regression
Robust Regression	Regression
Decision Tree Classification	Classification

2. Important basic concepts

Linear Regression

Linear Regression finds the relationship between input features and a numerical output.

For house-price prediction:

[$\text{Price} = b_0 + b_1(\text{Size}) + b_2(\text{Rooms}) + b_3(\text{Age})$]

Where:

- **Price** = predicted house price
- (b_0) = intercept
- (b_1, b_2, b_3) = coefficients

- Size, Rooms, Age = input features

Coefficient

A coefficient tells us how much a feature affects the prediction.

Example:

$$\text{Price} = 10 + 0.04(\text{Size}) + 3(\text{Rooms}) - 0.5(\text{Age})$$

Meaning:

- Bigger house size increases price
- More rooms increase price
- Older house decreases price

Overfitting

Overfitting means the model memorizes training data but performs poorly on new data.

Example:

$$\begin{aligned}\text{Training } R^2 &= 0.99 \\ \text{Testing } R^2 &= 0.62\end{aligned}$$

Underfitting

Underfitting means the model is too simple and cannot understand the pattern.

Example:

$$\begin{aligned}\text{Training } R^2 &= 0.50 \\ \text{Testing } R^2 &= 0.48\end{aligned}$$

Regularization

Regularization adds a penalty for large coefficients.

$$\text{Final loss} = \text{Prediction error} + \text{Penalty}$$

It helps reduce overfitting.

Ridge, Lasso and Elastic Net are regularization algorithms.

3. Common real-life dataset

Run this code first. We will use this simulated house-price data for Ridge, Lasso and Elastic Net.

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# Create repeatable random data
rng = np.random.default_rng(42)

number_of_houses = 300

size_sqft = rng.integers(600, 3000, number_of_houses)

# Rooms and size are related features
rooms = np.clip(
    np.round(size_sqft / 500 + rng.normal(0, 0.5, number_of_houses)),
    1,
    7
).astype(int)

age_years = rng.integers(0, 40, number_of_houses)
distance_km = rng.integers(1, 30, number_of_houses)
parking_spaces = rng.integers(0, 4, number_of_houses)

# This feature is useless for house-price prediction
random_house_number = rng.integers(1000, 9999, number_of_houses)

# Create simulated house prices in lakh
price_lakh = (
    10
    + 0.035 * size_sqft
    + 3.5 * rooms
    - 0.45 * age_years
    - 1.1 * distance_km
    + 2.5 * parking_spaces
    + rng.normal(0, 5, number_of_houses)
)

df = pd.DataFrame({
    "size_sqft": size_sqft,
    "rooms": rooms,
    "age_years": age_years,
    "distance_km": distance_km,
    "parking_spaces": parking_spaces,
```

```
"random_house_number": random_house_number,
"price_lakh": price_lakh
})

X = df.drop(columns="price_lakh")
y = df["price_lakh"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print(df.head())
```

Evaluation function

```
def evaluate_regression(model, X_test, y_test):
    predictions = model.predict(X_test)

    mae = mean_absolute_error(y_test, predictions)
    mse = mean_squared_error(y_test, predictions)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, predictions)

    print("MAE:", round(mae, 2))
    print("MSE:", round(mse, 2))
    print("RMSE:", round(rmse, 2))
    print("R2 score:", round(r2, 2))
```

4. Ridge Regression

Easy definition

Ridge is Linear Regression with an **L2 penalty**.

It makes large coefficients smaller but normally does not make them exactly zero.

Real-life example

Suppose these features are related:

- House size
- Number of rooms

- Covered area

A large house usually has more rooms and more covered area.

This relationship between input features is called **multicollinearity**.

Normal Linear Regression may give unstable coefficients when features are strongly related.

Ridge handles related features well. It keeps them but reduces their coefficient sizes.

Ridge formula

[Ridge\ Loss = $\text{MSE} + \alpha \sum_{j=1}^n b_j^2$]

Easy meaning:

Ridge loss =
Prediction mistakes + Square of every coefficient

Where:

- **MSE** = Mean Squared Error
- (b_j) = coefficient
- (α) = penalty strength

Why square the coefficients?

Large coefficients receive a bigger penalty.

Example:

Coefficient = 2 → Square = 4
Coefficient = 10 → Square = 100

Therefore, Ridge strongly controls large coefficients.

Ridge code

```
from sklearn.linear_model import Ridge

ridge_model = make_pipeline(
    StandardScaler(),
    Ridge(alpha=1.0)
)

ridge_model.fit(X_train, y_train)

evaluate_regression(
    ridge_model,
    X_test,
    y_test
)
```

Predicting a new house price

```
new_house = pd.DataFrame({
    "size_sqft": [1800],
    "rooms": [4],
    "age_years": [5],
    "distance_km": [7],
    "parking_spaces": [2],
    "random_house_number": [4587]
})

predicted_price = ridge_model.predict(new_house)

print(
    "Predicted price:",
    round(predicted_price[0], 2),
    "lakh"
)
```


Displaying coefficients

```
trained_ridge = ridge_model.named_steps["ridge"]

coefficient_table = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": trained_ridge.coef_
})

print(coefficient_table)
```

Ridge normally keeps every coefficient non-zero.

Important Ridge parameters

alpha

Controls the penalty.

```
Ridge(alpha=1.0)
```

Alpha	Result
0	Almost normal Linear Regression
Small	Weak penalty
Large	Strong penalty
Too large	Possible underfitting

Important interview point:

A larger **alpha** produces smaller coefficients.

fit_intercept

```
Ridge(fit_intercept=True)
```

- **True**: model calculates the intercept
- **False**: model does not calculate the intercept

Normally use **True**.

solver

Controls the mathematical method used for training.

```
Ridge(solver="auto")
```

Common values:

- **auto**: Scikit-learn chooses automatically
- **svd**: stable for small datasets
- **lsqr**: useful for large datasets
- **sag** and **saga**: useful for large datasets

max_iter

Maximum number of training steps.

```
Ridge(max_iter=10000)
```

tol

The model stops when improvement becomes very small.

```
Ridge(tol=0.0001)
```

Finding the best **alpha**

```
from sklearn.linear_model import RidgeCV

ridge_cv_model = make_pipeline(
    StandardScaler(),
    RidgeCV(
        alphas=[0.001, 0.01, 0.1, 1, 10, 100],
        cv=5
    )
)

ridge_cv_model.fit(X_train, y_train)

best_ridge = ridge_cv_model.named_steps["ridgecv"]

print("Best alpha:", best_ridge.alpha_)

evaluate_regression(
    ridge_cv_model,
    X_test,
    y_test
)
```

cv=5 means the training data is divided into five parts for validation.

Ridge advantages

- Reduces overfitting
- Handles related features well
- Makes the model more stable
- Keeps all input features
- Works well when most features are useful

Ridge disadvantages

- Does not remove useless features
- Feature scaling is important
- A very large **alpha** causes underfitting
- The relationship should be approximately linear

Interview answer

Ridge Regression is Linear Regression with an L2 penalty. It reduces large coefficients and overfitting but normally does not make coefficients exactly zero.

5. Lasso Regression

Full form

LASSO = Least Absolute Shrinkage and Selection Operator

Easy definition

Lasso is Linear Regression with an **L1 penalty**.

It can make some coefficients exactly zero. Therefore, it can automatically remove less useful features.

Real-life example

We added **random_house_number** to our dataset.

A random house number should not decide the house price.

Lasso may give it a zero coefficient:

Size coefficient	= 20.5
Rooms coefficient	= 4.2
Age coefficient	= -5.1
Random house number coefficient	= 0

The feature with coefficient zero is ignored.

Lasso formula

[Lasso\ Loss = $\text{MSE} + \alpha \sum_{j=1}^n |b_j|$]

Easy meaning:

Lasso loss =
Prediction mistakes + Absolute value of coefficients

What is absolute value?

$ -5 = 5$
$ 5 = 5$

Lasso uses the absolute value of each coefficient.

Lasso code

```
from sklearn.linear_model import Lasso

lasso_model = make_pipeline(
    StandardScaler(),
    Lasso(
        alpha=1.0,
        max_iter=10000
    )
)

lasso_model.fit(X_train, y_train)

evaluate_regression(
    lasso_model,
    X_test,
    y_test
)
```

Displaying selected and removed features

```
trained_lasso = lasso_model.named_steps["lasso"]

coefficient_table = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": trained_lasso.coef_
})

print(coefficient_table)

print("\nSelected features:")
print(
    coefficient_table[
        coefficient_table["Coefficient"] != 0
    ]
)

print("\nRemoved features:")
print(
    coefficient_table[
        coefficient_table["Coefficient"] == 0
    ]
)
```

Important Lasso parameters

alpha

Controls the L1 penalty.

```
Lasso(alpha=1.0)
```

- Small **alpha**: fewer features are removed
- Large **alpha**: more features may be removed
- Very large **alpha**: important features may also be removed

max_iter

Maximum training iterations.

```
Lasso(max_iter=10000)
```

Increase it if you get a convergence warning.

tol

Training stops when improvement becomes very small.

```
Lasso(tol=0.0001)
```

selection

Controls how coefficients are updated.

```
Lasso(selection="cyclic")
```

Options:

- **cyclic**: updates features in order
- **random**: updates features randomly

positive

```
Lasso(positive=True)
```

If **True**, all coefficients must be positive.

Use it only when negative coefficients are impossible in the real problem.

Finding the best **alpha**

```
from sklearn.linear_model import LassoCV

lasso_cv_model = make_pipeline(
    StandardScaler(),
    LassoCV(
        alphas=[0.001, 0.01, 0.1, 0.5, 1, 5, 10],
        cv=5,
        max_iter=10000
    )
)

lasso_cv_model.fit(X_train, y_train)

best_lasso = lasso_cv_model.named_steps["lassocv"]

print("Best alpha:", best_lasso.alpha_)
print("Coefficients:", best_lasso.coef_)

evaluate_regression(
    lasso_cv_model,
    X_test,
    y_test
)
```

Lasso advantages

- Reduces overfitting
- Automatically selects features
- Removes less useful features
- Makes the model simpler
- Useful when the dataset has many columns

Lasso disadvantages

- May remove an important feature
- May behave poorly when features are strongly related
- Scaling is important

- Large **alpha** can cause underfitting

Interview answer

Lasso Regression is Linear Regression with an L1 penalty. It can make less useful feature coefficients exactly zero, so it performs feature selection.

6. Elastic Net Regression

Easy definition

Elastic Net combines Ridge and Lasso.

[Elastic\ Net = Ridge + Lasso]

It can:

- Reduce coefficients like Ridge
- Remove useless features like Lasso
- Handle related features better than Lasso

Real-life example

Suppose we have:

- House size
- Covered area
- Number of rooms
- Number of bedrooms

These features are related.

Lasso may select only one related feature and remove others.

Ridge keeps all of them.

Elastic Net finds a balance:

- Keeps useful related features
- Removes useless features
- Reduces large coefficients

Elastic Net formula

[Loss = MSE + $\alpha \left[\sum |b_j| \right] + \frac{1-r}{2} \sum b_j^2 \right]$

Here, (r) is **l1_ratio**.

Easy meaning:

Elastic Net loss =
Prediction error + Lasso penalty + Ridge penalty

Understanding **l1_ratio**

l1_ratio Meaning

1.0	Only Lasso-style penalty
------------	--------------------------

0.8	More Lasso, less Ridge
------------	------------------------

0.5	Equal mixture
------------	---------------

0.2	Less Lasso, more Ridge
------------	------------------------

0.0	Only Ridge-style penalty
------------	--------------------------

Elastic Net code

```
from sklearn.linear_model import ElasticNet

elastic_model = make_pipeline(
    StandardScaler(),
    ElasticNet(
        alpha=1.0,
        l1_ratio=0.5,
        max_iter=10000
    )
)

elastic_model.fit(X_train, y_train)

evaluate_regression(
    elastic_model,
    X_test,
    y_test
)
```

Displaying coefficients

```
trained_elastic = elastic_model.named_steps["elasticnet"]

coefficient_table = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": trained_elastic.coef_
})

print(coefficient_table)
```

Important Elastic Net parameters

alpha

Controls the total penalty.

```
ElasticNet(alpha=1.0)
```

- Large **alpha**: strong penalty
- Small **alpha**: weak penalty

l1_ratio

Controls the Lasso and Ridge mixture.

```
ElasticNet(l1_ratio=0.5)
```

- Near **1**: more Lasso
- Near **0**: more Ridge

max_iter

```
ElasticNet(max_iter=10000)
```

Maximum number of training iterations.

tol

```
ElasticNet(tol=0.0001)
```

Controls the stopping condition.

selection

```
ElasticNet(selection="cyclic")
```

Controls coefficient update order.

Finding the best parameters

```
from sklearn.linear_model import ElasticNetCV

elastic_cv_model = make_pipeline(
    StandardScaler(),
    ElasticNetCV(
        alphas=[0.001, 0.01, 0.1, 0.5, 1, 5, 10],
        l1_ratio=[0.1, 0.3, 0.5, 0.7, 0.9],
        cv=5,
        max_iter=10000
    )
)

elastic_cv_model.fit(X_train, y_train)

best_elastic = elastic_cv_model.named_steps["elasticnetcv"]

print("Best alpha:", best_elastic.alpha_)
print("Best l1_ratio:", best_elastic.l1_ratio_)

evaluate_regression(
    elastic_cv_model,
    X_test,
    y_test
)
```

Elastic Net advantages

- Combines Ridge and Lasso
- Reduces overfitting
- Performs feature selection
- Handles related features better than Lasso
- Useful for datasets with many features

Elastic Net disadvantages

- Has more parameters

- Takes more time to tune
- Scaling is important
- Poor parameter values can cause underfitting

Interview answer

Elastic Net combines the L1 penalty of Lasso and the L2 penalty of Ridge. It can select features and also handle strongly related features.

7. Why is feature scaling important?

Suppose:

House size = 2,000
Rooms = 4

The values have very different ranges.

Regularization checks coefficient sizes. Without scaling, the penalty may treat the features unfairly.

That is why we use:

```
make_pipeline(  
    StandardScaler(),  
    Ridge()  
)
```

StandardScaler changes features to a similar scale.

Important interview point:

Scaling is important for Ridge, Lasso and Elastic Net, but normally not required for Decision Trees.

8. Ridge vs Lasso vs Elastic Net

Point	Ridge	Lasso	Elastic Net
Penalty	L2	L1	L1 + L2
Reduces coefficients	Yes	Yes	Yes

Point	Ridge	Lasso	Elastic Net
Makes coefficients zero	Normally no	Yes	Yes
Feature selection	No	Yes	Yes
Handles related features	Good	May select only one	Very good
Main parameters	alpha	alpha	alpha, l1_ratio
Scaling needed	Yes	Yes	Yes

Easy memory trick

- **Ridge:** Reduces coefficients
- **Lasso:** Leaves only useful features
- **Elastic Net:** Uses both methods

9. Robust Regression

Easy definition

Robust Regression is designed to reduce the effect of outliers.

What is an outlier?

An outlier is a value that is very different from most values.

Example:

Experience: 1, 2, 3, 4, 5 years
Salary: 3, 4, 5, 6, 50 lakh

50 lakh is an outlier.

It may be:

- A typing mistake
- A company director
- A special employee
- A real but rare value

Real-life example

A company wants to predict salary using experience.

One director earns much more than normal employees. Normal Linear Regression may move its line toward the director's salary.

Robust Regression reduces the effect of that extreme salary.

10. Huber Regression

Easy definition

Huber Regression uses:

- Squared error for small mistakes
- Linear error for very large mistakes

It does not completely remove outliers. It reduces their effect.

Huber loss formula

For a small error:

$$[L(e) = \frac{1}{2}e^2]$$

For a large error:

$$[L(e) = \delta \left(|e| - \frac{1}{2}\delta \right)]$$

Where:

$$[e = \text{Actual} - \text{Predicted}]$$

Easy meaning:

Small errors receive normal punishment, but extremely large errors do not receive an unnecessarily huge punishment.

Real-life Huber code

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import (
    LinearRegression,
    HuberRegressor
)

# Years of experience
X_salary = np.array([
    [1], [2], [3], [4], [5],
    [6], [7], [8], [9], [10]
])

# Salary in lakh
# 50 lakh is an outlier
y_salary = np.array([
    3, 4, 5, 6, 7,
    8, 9, 10, 11, 50
])

linear_model = LinearRegression()
linear_model.fit(X_salary, y_salary)

huber_model = HuberRegressor(
    epsilon=1.35,
    alpha=0.0001,
    max_iter=1000
)
huber_model.fit(X_salary, y_salary)

new_employee = np.array([[6]])

print(
    "Linear Regression salary:",
    linear_model.predict(new_employee)[0]
)

print(
    "Huber Regression salary:",
    huber_model.predict(new_employee)[0]
)
```

Visual comparison

```
X_line = np.linspace(1, 10, 100).reshape(-1, 1)

linear_line = linear_model.predict(X_line)
huber_line = huber_model.predict(X_line)

plt.scatter(
    X_salary,
    y_salary,
    color="black",
    label="Employees"
)

plt.plot(
    X_line,
    linear_line,
    color="red",
    label="Linear Regression"
)

plt.plot(
    X_line,
    huber_line,
    color="green",
    label="Huber Regression"
)

plt.xlabel("Years of experience")
plt.ylabel("Salary in lakh")
plt.title("Linear vs Huber Regression")
plt.legend()
plt.show()
```

Important Huber parameters

epsilon

Controls when a point is treated as an outlier.

```
HuberRegressor(epsilon=1.35)
```

- Smaller **epsilon**: more points are treated as outliers
- Larger **epsilon**: model behaves more like Linear Regression

- It must be greater than or equal to 1

alpha

Regularization strength.

```
HuberRegressor(alpha=0.0001)
```

max_iter

Maximum training steps.

tol

Stopping condition.

fit_intercept

Whether to calculate an intercept.

11. RANSAC Regression

Full form

RANSAC = Random Sample Consensus

Easy working

RANSAC:

1. Randomly selects some points.
2. Trains a regression line.
3. Checks which points are close to the line.
4. Close points are called **inliers**.
5. Far points are called **outliers**.
6. Repeats this process.

7. Keeps the best model.

RANSAC code

```
from sklearn.linear_model import RANSACRegressor

ransac_model = RANSACRegressor(
    estimator=LinearRegression(),
    min_samples=5,
    residual_threshold=2.0,
    max_trials=100,
    random_state=42
)

ransac_model.fit(X_salary, y_salary)

prediction = ransac_model.predict([[6]])

print("Predicted salary:", prediction[0])
print("Inliers:", ransac_model.inlier_mask_)
print("Outliers:", ~ransac_model.inlier_mask_)
```

Important RANSAC parameters

estimator

The base model:

```
estimator=LinearRegression()
```

min_samples

Minimum samples used in one random trial.

residual_threshold

Maximum error allowed for an inlier.

```
residual_threshold=2.0
```

If a point's error is greater than this value, it may be considered an outlier.

max_trials

Maximum number of random attempts.

random_state

Makes the result repeatable.

When should we use RANSAC?

Use it when:

- Data has strong outliers
 - Most data follows one clear pattern
 - You want to identify inliers and outliers
-

12. Theil-Sen Regression

Easy definition

Theil-Sen calculates regression slopes using many pairs of points and selects a middle or median result.

The median is less affected by extreme values.

Code

```
from sklearn.linear_model import TheilSenRegressor

theil_model = TheilSenRegressor(
    random_state=42
)

theil_model.fit(X_salary, y_salary)

prediction = theil_model.predict([[6]])

print("Predicted salary:", prediction[0])
```

Advantages

- Resistant to outliers
- Good for small or medium datasets
- Does not depend strongly on extreme values

Disadvantages

- Can be slow on large datasets
 - More expensive than Linear Regression
-

13. Robust Regression comparison

Method	Outlier handling	Best situation
Huber	Reduces outlier effect	Moderate outliers
RANSAC	Separates outliers from normal points	Strong outliers
Theil-Sen	Uses a median-based slope	Small datasets

Interview point

Ridge and Lasso reduce overfitting, but they are not specially designed for outliers. Robust Regression is specially designed to reduce the effect of outliers.

14. Decision Tree Classification

Easy definition

Decision Tree Classification predicts a category by asking a series of questions.

It works like a flowchart.

Real-life loan example

```

graph TD
    A[Is credit score greater than 700?] -- Yes --> B[Is income greater than ₹50,000?]
    A -- No --> C[No → Loan rejected]
    B -- Yes --> D[Yes → Loan approved]
    B -- No --> E[No → Loan rejected]
  
```

Important tree terms

Root node

The first question at the top.

Credit score > 700?

Decision node

A middle question.

Income > ₹50,000?

Branch

The path produced by an answer.

Yes branch
No branch

Leaf node

The final prediction.

Approved
Rejected

Split

Dividing data using a condition.

Parent node

A node before division.

Child node

A node created after a split.

Depth

The number of levels in the tree.

Pure node

A node containing only one class.

10 approved, 0 rejected

15. How does a tree select the best question?

The tree tries questions like:

Credit score ≤ 680 ?
 Income $\leq 50,000$?
 Previous default ≤ 0 ?
 Existing EMI $\leq 15,000$?

It selects the question that separates classes most clearly.

Common splitting methods:

- Gini impurity
- Entropy
- Information gain

16. Gini impurity

Formula

$$[\text{Gini} = 1 - \sum_{i=1}^k p_i^2]$$

Where (p_i) is the percentage of a class.

Simple example

A node contains:

8 approved
 2 rejected

Probabilities:

$$[P(\text{Approved}) = \frac{8}{10} = 0.8]$$

$$[P(\text{Rejected}) = \frac{2}{10} = 0.2]$$

Gini:

$$[\text{Gini} = 1 - (0.8^2 + 0.2^2)]$$

$$[\text{Gini} = 1 - (0.64 + 0.04) = 0.32]$$

Meaning:

- **Gini = 0**: completely pure
- Higher Gini: more mixed data

17. Entropy

Formula

$$[\text{Entropy} = -\sum_{i=1}^k p_i \log_2(p_i)]$$

Meaning:

- **Entropy = 0**: completely pure
- High entropy: classes are mixed

Gini vs Entropy

Gini	Entropy
Faster to calculate	Slightly slower
Default in Scikit-learn	Uses information theory
Usually gives similar results	Usually gives similar results

18. Information gain

Information gain tells us how much a split improved purity.

$$[\text{Gain} = I(\text{Parent}) - \left[\frac{N_L}{N} I(\text{Left}) + \frac{N_R}{N} I(\text{Right}) \right]]$$

Where:

- I = impurity
- N_L = left-side samples
- N_R = right-side samples
- N = total samples

Easy meaning:

Information gain =
Impurity before split - Impurity after split

The tree prefers higher information gain.

19. Real-life loan dataset code

This is simulated learning data, not a real bank policy.

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

rng = np.random.default_rng(42)

number_of_customers = 600

income = rng.integers(
    20000,
    150000,
    number_of_customers
)

credit_score = rng.integers(
    300,
    851,
    number_of_customers
)

existing_emi = rng.integers(
    0,
    50000,
    number_of_customers
)

previous_default = rng.integers(
    0,
    2,
    number_of_customers
)

# Create a simple simulated approval rule
loan_approved = (
    (credit_score >= 680)
    & ((income - existing_emi) >= 35000)
    & (previous_default == 0)
).astype(int)

# Add a little noise because real-life data is not perfect
noise_indices = rng.choice(
    number_of_customers,
    size=30,
```



```
        replace=False
    )

    loan_approved[noise_indices] = (
        1 - loan_approved[noise_indices]
    )

    loan_df = pd.DataFrame({
        "income": income,
        "credit_score": credit_score,
        "existing_emi": existing_emi,
        "previous_default": previous_default,
        "loan_approved": loan_approved
    })

    X = loan_df.drop(columns="loan_approved")
    y = loan_df["loan_approved"]

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.20,
        random_state=42,
        stratify=y
    )

    tree_model = DecisionTreeClassifier(
        criterion="gini",
        max_depth=4,
        min_samples_split=10,
        min_samples_leaf=5,
        random_state=42
    )

    tree_model.fit(X_train, y_train)

    predictions = tree_model.predict(X_test)

    print(
        "Accuracy:",
        accuracy_score(y_test, predictions)
    )

    print("\nConfusion matrix:")
    print(confusion_matrix(y_test, predictions))

    print("\nClassification report:")
    print(classification_report(y_test, predictions))
```

20. Predicting a new customer

```
new_customer = pd.DataFrame({
    "income": [75000],
    "credit_score": [740],
    "existing_emi": [10000],
    "previous_default": [0]
})

prediction = tree_model.predict(new_customer)
probability = tree_model.predict_proba(new_customer)

if prediction[0] == 1:
    print("Loan approved")
else:
    print("Loan rejected")

print("Rejection probability:", probability[0][0])
print("Approval probability:", probability[0][1])
```

21. Important Decision Tree parameters

criterion

Selects the impurity measurement.

```
criterion="gini"
```

Options:

- **gini**
- **entropy**
- **log_loss**

splitter

Controls how the split is selected.

```
splitter="best"
```

Options:

- **best**: selects the best split

- **random**: selects from random splits

max_depth

Maximum number of tree levels.

```
max_depth=4
```

- Small value: simpler tree
- Large value: more complex tree
- Too large: possible overfitting

min_samples_split

Minimum records required to split a node.

```
min_samples_split=10
```

A node with fewer than 10 records will not split.

min_samples_leaf

Minimum records required in a final leaf.

```
min_samples_leaf=5
```

Larger values usually make the tree simpler.

max_features

Maximum features checked during a split.

```
max_features=None
```

Options include:

- **None**: check all features
- **"sqrt"**: check square root of total features
- **"log2"**: check log base 2 of total features

max_leaf_nodes

Maximum final leaves.

```
max_leaf_nodes=10
```

class_weight

Helpful for unbalanced classes.

```
class_weight="balanced"
```

Example:

```
950 rejected  
50 approved
```

The classes are unbalanced. **balanced** gives more importance to the smaller class.

ccp_alpha

Controls post-pruning.

```
ccp_alpha=0.01
```

- **0**: no pruning
- Larger value: removes more branches

random_state

Makes results repeatable.

```
random_state=42
```

22. Overfitting and pruning

A large tree may memorize training data.

```
Training accuracy = 100%  
Testing accuracy = 72%
```

This is overfitting.

Pre-pruning

Stop the tree before it grows too much.

```
DecisionTreeClassifier(  
    max_depth=4,  
    min_samples_split=10,  
    min_samples_leaf=5,  
    max_leaf_nodes=15  
)
```

Post-pruning

Grow the tree and then remove weak branches.

```
DecisionTreeClassifier(  
    ccp_alpha=0.01  
)
```

23. Displaying tree rules

```
from sklearn.tree import export_text  
  
tree_rules = export_text(  
    tree_model,  
    feature_names=list(X.columns)  
)  
  
print(tree_rules)
```

Possible rules:

```
credit_score <= 679.5  
|--- loan rejected  
  
credit_score > 679.5  
|--- previous_default <= 0.5  
| |--- income <= 50000  
| | |--- loan rejected  
| |--- income > 50000  
| | |--- loan approved
```

24. Visualizing the tree

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(18, 10))

plot_tree(
    tree_model,
    feature_names=X.columns,
    class_names=["Rejected", "Approved"],
    filled=True,
    rounded=True
)

plt.show()
```

25. Feature importance

Feature importance shows which features the tree used most.

```
importance_table = pd.DataFrame({
    "Feature": X.columns,
    "Importance": tree_model.feature_importances_
})

importance_table = importance_table.sort_values(
    by="Importance",
    ascending=False
)

print(importance_table)
```

Important interview point:

Feature importance shows usefulness for prediction. It does not prove that the feature caused the result.

26. Finding the best tree parameters

```
from sklearn.model_selection import GridSearchCV

parameter_grid = {
    "criterion": ["gini", "entropy"],
    "max_depth": [2, 3, 4, 5, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 5],
    "ccp_alpha": [0.0, 0.001, 0.01]
}

search = GridSearchCV(
    estimator=DecisionTreeClassifier(
        random_state=42
    ),
    param_grid=parameter_grid,
    cv=5,
    scoring="f1",
    n_jobs=-1
)

search.fit(X_train, y_train)

print("Best parameters:", search.best_params_)
print("Best validation score:", search.best_score_)

best_tree = search.best_estimator_

test_predictions = best_tree.predict(X_test)

print(
    "Testing accuracy:",
    accuracy_score(y_test, test_predictions)
)
```

27. Classification evaluation metrics

Accuracy

Out of all predictions, how many were correct?

[$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$]

Precision

Out of all customers predicted as approved, how many were actually approved?

$$[\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}]$$

Recall

Out of all actually approved customers, how many did the model find?

$$[\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}]$$

F1-score

Balance between Precision and Recall.

$$[F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}]$$

Confusion matrix

	Predicted Reject	Predicted Approve
Actual Reject	True Negative	False Positive
Actual Approve	False Negative	True Positive

28. Decision Tree advantages

- Easy to understand
- Works like a real-life flowchart
- Does not normally require scaling
- Handles non-linear relationships
- Can be used for classification and regression
- Gives feature importance
- Can handle multiple classes

29. Decision Tree disadvantages

- Can overfit easily
 - A small data change can create a different tree
 - Deep trees are difficult to understand
 - One tree may be less accurate than Random Forest
 - Biased training data can create unfair predictions
-

30. Most important interview differences

Ridge vs Lasso

Ridge	Lasso
Uses L2 penalty	Uses L1 penalty
Reduces coefficients	Reduces coefficients
Normally does not make them zero	Can make them zero
Keeps all features	Performs feature selection
Good for related features	Good when many features are useless

Lasso vs Elastic Net

Lasso	Elastic Net
Only L1 penalty	L1 and L2 penalties
May choose only one related feature	Can keep multiple related features
One main parameter	Two main parameters
alpha	alpha and l1_ratio

Ridge vs Robust Regression

Ridge	Robust Regression
Reduces overfitting	Reduces outlier effect
Uses coefficient penalty	Uses special outlier-resistant loss
Not specially made for outliers	Specially made for outliers

Decision Tree vs Logistic Regression

Decision Tree	Logistic Regression
Uses question-based splits	Uses a mathematical equation

Decision Tree	Logistic Regression
Scaling normally not required	Scaling is recommended
Handles non-linear patterns	Mainly learns a linear boundary
Can overfit easily	Usually more stable
Easy rules	Easy coefficients

31. Top interview questions and answers

1. What is regularization?

Regularization adds a penalty to large coefficients to reduce model complexity and overfitting.

2. What is Ridge Regression?

Ridge is Linear Regression with an L2 penalty. It reduces coefficient size but normally keeps all features.

3. What is Lasso Regression?

Lasso is Linear Regression with an L1 penalty. It can make less useful feature coefficients zero.

4. What is Elastic Net?

Elastic Net combines Ridge and Lasso penalties.

5. What does **alpha** do?

alpha controls regularization strength. A larger **alpha** gives a stronger penalty.

6. What does **l1_ratio** do?

It controls the Lasso and Ridge mixture in Elastic Net.

7. Which algorithm performs feature selection?

Lasso and Elastic Net can perform feature selection.

8. Which algorithm handles multicollinearity?

Ridge and Elastic Net handle strongly related features well.

9. What happens when **alpha** is too large?

Coefficients become too small and the model may underfit.

10. Why is scaling required?

Because regularization compares coefficient sizes. Features should be on similar scales.

11. What is Robust Regression?

It is regression designed to reduce the effect of outliers.

12. What is Huber Regression?

Huber uses squared loss for normal errors and a smaller linear loss for large errors.

13. What is RANSAC?

RANSAC repeatedly trains on random samples and separates normal points from outliers.

14. What is a Decision Tree?

A Decision Tree predicts an answer by asking feature-based questions.

15. What is Gini impurity?

Gini measures how mixed the classes are inside a node. A lower Gini is better.

16. What is entropy?

Entropy measures disorder or class mixing inside a node.

17. What is information gain?

It measures how much a split reduces impurity.

18. Does a Decision Tree require scaling?

No, because it uses value-based split conditions.

19. Why does a Decision Tree overfit?

An unrestricted tree can create many small branches and memorize training data.

20. How can we control Decision Tree overfitting?

Use:

- `max_depth`
 - `min_samples_split`
 - `min_samples_leaf`
 - `max_leaf_nodes`
 - `ccp_alpha`
-

32. Final selection guide

Most features are useful

→ Ridge

Many features are useless

→ Lasso

Features are related and some are useless

→ Elastic Net

Data contains outliers

→ Huber, RANSAC or Theil-Sen

Need to predict a category using rules

→ Decision Tree Classification

Final one-line memory

Ridge → Shrinks all coefficients

Lasso → Can remove features

Elastic Net → Ridge + Lasso

Robust → Handles outliers

Tree → Predicts by asking questions